UNIT-V
FUNCTIONS & BASICS OF FILE HANDLING

INTRODUCTION

A function in C is a set of statements that when called perform some specific tasks. It is the basic building block of a C program that provides modularity and code reusability. The programming statements of a function are enclosed within { } braces, having certain meanings and performing certain operations. They are also called subroutines or procedures in other languages.

Advantages of using Functions

- Functions avoid rewriting the same logic/code again and again in a program.
- Functions can be called any number of times in a program and from any place in a program.
- A C program can be tracked easily when it is divided into multiple functions.
- Reusability is the main achievement of C functions.

Function Aspects

There are three aspects of a C function. They are -

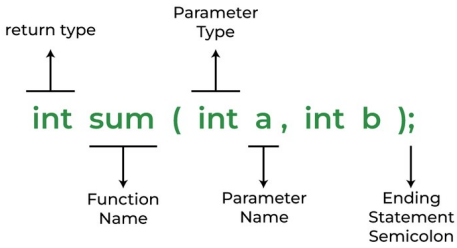
- Function Declaration
- Function Definition
- Function Call

FUNCTION DECLARATION - A function must be declared globally in a c program to tell the compiler about the function name, function parameters, and return type.

Syntax → return_type name_of_the_function (*Arguments_List*);

The parameter name is not mandatory when declaring functions.

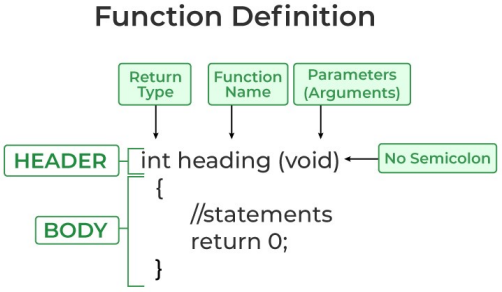
Example → int sum(int a, int b);
int max(int , int);



FUNCTION DEFINITION - It contains the actual statements which are to be executed. It is the most important aspect to which the control comes when the function is called. Only one value can be returned from the function.

Syntax →

return_type function_name (*Arguments_List*)
{
// body of the function
}



A function definition in C programming consists of a function header and a function body. Here are all the parts of a function −

Return Type − A function may return a value. The return_type is the data type of the value the function returns. Some functions perform the desired operations without returning a value. In this case, the return_type is the keyword void.

Function Name − This is the actual name of the function. The function name and the parameter list together constitute the function signature.

Arguments – An argument is like a placeholder. When a function is invoked, value is passed to the argument. This value is referred to as actual parameter or argument. The argument list refers to the type, order, and number of the arguments of a function. Arguments are optional; that is, a function may contain no arguments.

Function Body − The function body contains a collection of statements that define what the function does.

Function call Function can be called from anywhere in the program. The parameter list must not differ in function calling and function declaration. We must pass the same number of functions as it is declared in the function declaration.

Example - Given below is the source code or definition for a function called **max().** This function takes two parameters num1 and num2 and returns the maximum value between the two −

```
void max(int num1, int num2)
{
    int result;
   if (num1 > num2)
     result = num1;
  else
     result = num2;
   return result;
}
```

FUNCTION CALL - To use a defined function, that function must be called to perform the defined task. This is known as Function calling.

When a program calls a function, the program control is transferred to the called function. A called function performs a defined task and when its return statement is executed or when its function-ending closing brace is reached, it returns the program control back to the calling function.

A function can be called by specifying the function and passing the required arguments. If the function is returning a value then it is to be received in calling function.

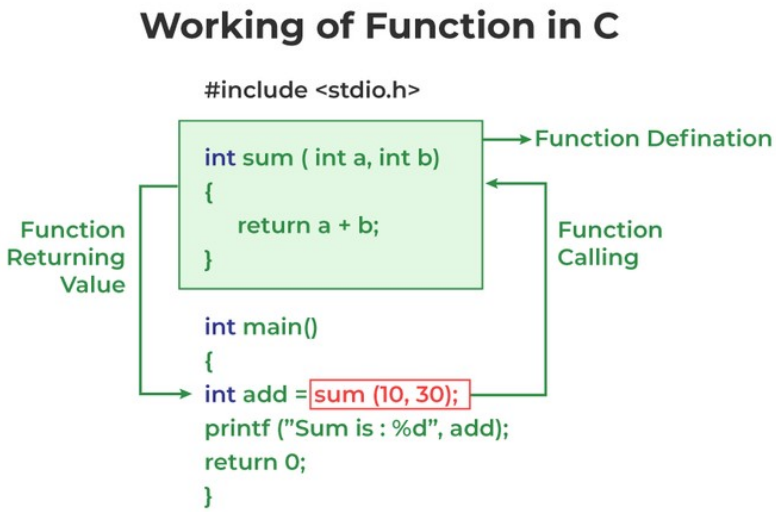
Syntax → function_name (argument_list)

Example → max(a,b)
max(24,b);

Example Program using Functions.

```
// function declaration
int max(int, int);
void main()
{
        int x=20, y=40, mx;
        // function calling
        mx = max(x, y);
        printf( "Max value is : %d", mx );
}
// function definition
int max(int num1, int num2)
{
        int result;
        if (num1 > num2)
            result = num1;
        else
            result = num2;
        return result;
}
```

Output Max value is : 40



FUNCTION RETURN TYPES AND ARGUMENTS

In C programming language, functions can be called either with or without arguments and might return values. They may or might not return values to the calling functions. Therefore, functions can be differentiated into 4 types according to the arguments passed and value returns these are:

- Function with no arguments and return value.
- Function with no arguments and with return value
- Function with argument and no return value
- Function with arguments and return value.

An example general form of Function with no arguments and return value
Function declaration : void function(**void**);
Function call : function();
Function definition :
void function(**void**)
{
statements;
// no return statement
}
	An example general form of Function with no arguments and with return value
	Function declaration : **int** function(**void**);
	Function call : function();
	Function definition :
	int function(**void**)
	{
	statements;
	return x;
	}

An example general form of Function with arguments and no return value
Function declaration : **void** function (**int**);
Function call : function(**x**);
Function definition:
void function(**int x**)
{
statements;
}
	An example general form of Function with arguments and return value
	Function declaration : **int** function (**int**);
	Function call : function(**x**);
	Function definition:
	int function(**int x**)
	{
	statements;
	return x;
	}

PARAMETER (ARGUMENT) PASSING TECHNIQUES

When a function needs to use the arguments, one must declare those variables that happen to accept the arguments' values in function header. Such variables are known as the given function's formal parameters or arguments. During the calling of a function, there will be two ways in which we can perform the passing of these arguments to a given function: 1) Call by Value or Pass by Value
2) Call by Reference or Pass by Reference

1) Call by Value - Parameter passing in this method copies values from actual parameters into formal function parameters. As a result, any changes made inside the functions do not reflect in the calling function.

Example

```
void swap(int n1, int n2)
{
 int temp = n1;
 n1 = n2;
 n2 = temp;
}
 void main()
{
 int x = 3, y = 2;
 printf("Before swap\n");
 printf("X=%d Y= %d\n", x, y);
 swap(x, y); // Values passed
 printf("After swap\n");
 printf("X=%d Y= %d\n", x, y);
}
```

Output
Before swap
X=3 Y=2
After swap
X=3 Y=2

2) Call by Reference – In this method a function is called by passing address as arguments instead of values and pointers are used as formal parameters to refer the addresses of actual arguments. Hence in this case, the changes performed in the values inside the called function can be directly reflected in the calling function.

Example

```
void swap(int *p1, int *p2)
{
 int temp = *p1;
 *p1 = *p2;
 *p 2 = temp;
}
 void main()
{
 int x = 3, y = 2;
 printf("Before swap\n");
 printf("X=%d Y= %d\n", x, y);
 swap(&x, &y); // Addresses passed
 printf("After swap\n");
 printf("X=%d Y= %d\n", x, y);
}
```

Output
Before swap
X=3 Y=2
After swap
X=2 Y=3

PASSING ARRAYS TO FUNCTIONS

In C language, it is not possible to pass all the array element values as a whole to a function. Otherwise, it is allowed to pass a reference or pointer (base address of the array) to the array. In this case, there are three ways to declare formal parameter in function definition. These three ways are described below.

- Way-1: Formal parameter as an unsized array - **return_type function(type arrayname[ ])**
- Way-2: Formal parameter as an sized array - **return_type function(type arrayname[size_of_array])**
- Way-3: Formal parameter as a pointer - **return_type function(type *ptr_Name)**

Example

Now, consider the following function, which takes an array as an argument along with another argument and based on the passed arguments, it returns the sum of all the numbers passed through the array.

```
int sumArray(int[],int);
void main()
{
   int arr[]={6, 8, 3, 5, 9}, as;
   as = sumArray(arr, 5);
  printf("Sum of Array = %d", as);
}
int sumArray(int x[], int n)
{
   int i, sm = 0;
   for(i=0;i<n;i++)
      sm = sm + x[i];
   return sm;
}
```

Output
Sum of Array = 31

SCOPE AND LIFETIME OF VARIABLES

The **scope of a variable** refers to which different parts of a program have access to the variable, that is, where the variable is visible. Here the term variable refers to all C data types: simple variables, arrays, structure, pointers, etc.

The **lifetime of a variable** refers that how long the variable persists in memory, or when the variable's storage is allocated and de-allocated.

Types of Variables Based on Scope

The variables are broadly classified as **local** and **global** based on scope.

Local Variables - The variable that is declared in a function or code is called a local variable. The scope of Local variables is within the defined function only. You cannot use a local variable outside the function (in which it is declared).
Global Variables - The scope of the global variable is the entire program. They are not defined inside any function and can be used in any function.

STORAGE CLASSES
Depending upon the scope and lifetime of variables, C has its four storage classes for variables used in any functions.

- Automatic variables
- Static variables
- Register variables
- External variables

Automatic Variables - All variables declared in C programming are automatic by default. **auto** keyword is used to declare automatic variables. An automatic variable is a local variable whose lifetime is within the code only. Its default value is garbage.
Syntax: auto data_type variable_name = initial_value; Or data_type variable_name = initial value;

Static Variables - The static variable is defined using the static keyword. Its scope depends on the area of its declaration. If a static variable is defined within a function, it is a local variable. If declared outside the function, its scope is global.
A static variable statically allocated memory to the variable and its lifetime throughout the program. It holds its value whenever a program is called. The default value of the static variable is 0.
Syntax: static data_type variable_name = initial_value

Register Variables - Register variable uses register keyword before its definitions, and it is stored in the CPU register. It is a local variable. These variables have fast processing. The default value of the register variable is garbage. Its lifetime is till the end of the function.
Syntax: register data_type variable_name = initial_value

External Variables - The extern variables use the extern keyword before the variable definition. This enhances the variable visibility, and the variable can be used in different C files. It is a global variable. It is not necessary to initialize the variable at the time of its declaration. Its lifetime is the entire program.
Syntax: extern data_type variable_name;
Example: extern int a;

Example program illustrating scope and storage classes of variables.

```
void fun();
void main()
{
   int j; // local variable
   register int z = 30; // register variable
   extern int i; // accessing external variable
   printf("Value of Register Variable = %d\n", z);
   printf("Value of External Variable = %d\n", i);
   for(j=0;j<4;j++)
   {
         printf("Function Call-%d\n",j+1);
         fun();
   }
}
int i = 100; // global variable
void fun()
{
   int x = 10; // auto variable
   static int y = 20; // static variable
   printf("Value of Local Variable = %d\n", x);
   printf("Value of Static Variable = %d\n", y);
   x = x + 10;
   y = y + 10;
}
```

Output

```
Value of Register Variable = 30
Value of External Variable = 100
Function Call-1
Value of Local Variable = 10
Value of Static Variable = 20
Function Call-2
Value of Local Variable = 10
Value of Static Variable = 30
Function Call-3
Value of Local Variable = 10
Value of Static Variable = 40
Function Call-4
Value of Local Variable = 10
Value of Static Variable = 50
```

BASICS OF FILE HANDLING IN C

File handing in C is the process in which we create, open, read, write, and close operations on a file. C language provides different functions such as fopen(), fwrite(), fread(), fseek(), fprintf(), etc. to perform input, output, and many different C file operations in our program.

Advantages of using Files

- Reusability: The data stored in the file can be accessed, updated, and deleted anywhere and anytime providing high reusability.
- Portability: Without losing any data, files can be transferred to another in the computer system. The risk of flawed coding is minimized with this feature.
- Efficient: A large amount of input may be required for some programs. File handling allows you to easily access a part of a file using few instructions which saves a lot of time and reduces the chance of errors.
- Storage Capacity: Files allow you to store a large amount of data without having to worry about storing everything simultaneously in a program.

TYPES OF FILES IN C

A **File** is a collection of data stored in the secondary memory. A file can be classified into two types based on the way the file stores the data. They are as follows: 1) Text File 2) Binary File

1. Text File - A text file contains data in the form of ASCII characters and is generally used to store a stream of characters.

- Each line in a text file ends with a new line character ('\n').
- It can be read or written by any text editor.
- They are generally stored with .txt file extension.
- Text files can also be used to store the source code.

2. Binary File - A binary file contains data in binary form (i.e. 0's and 1's) instead of ASCII characters. They contain data that is stored in a similar manner to how it is stored in the main memory.

- Binary files can be created only from within a program and their contents can only be read by a program.
- More secure as they are not easily readable.
- They are generally stored with .bin file extension.

File Pointer in C

A file pointer is a reference to a particular position in the opened file. It is used in file handling to perform all file operations such as read, write, close, etc. We use the FILE macro to declare the file pointer variable. The FILE macro is defined inside <stdio.h> header file.

Syntax of File Pointer → FILE* pointer_name;

File Pointer is used in almost all the file operations in C.

C FILE OPERATIONS

C file operations refer to the different possible operations that we can perform on a file in C such as:

- Creating or opening file
- Reading from file
- Writing to a file
- Closing a file

Create/Open a File - For opening a file in C, the fopen() function is used with the filename or file path along with the required access modes. If the file is opened successfully, fopen() returns a file pointer to it. Otherwise, returns NULL.

Syntax → fopen (File_Name, Access_Mode)

File_Name: name of the file to oepn.

Access_Mode: Specifies for what operation the file is being opened.

File access modes

File opening modes or access modes specify the allowed operations on the file to be opened. Some of the commonly used file access modes are listed below:

Opening Modes	Description
r	Open an existing file for reading in text mode. If the file cannot be opened returns NULL.
rb	Open an existing file for reading in binary mode. If the file cannot be opened returns NULL.
w	Open for writing in text mode. If the file exists, its contents are overwritten. If the file doesn't exist, a new file is created. Returns NULL, if unable to open the file.

Opening Modes	Description
wb	Open for writing in binary mode. If the file exists, its contents are overwritten. If the file does not exist, it will be created.
a	If the file is opened successfully, it sets up file pointer to the last character in it for append in text mode. If the file doesn't exist, a new file is created. Returns NULL, if unable to open the file.
ab	Open to append in binary mode. Data is added to the end of the file. If the file does not exist, it will be created.

Reading From a File - The file read operation in C can be performed using functions fscanf() or fgets().
fscanf() is used to read formatted string and variable arguments list to take input from a file.
fgets() is used to Input the whole line from the file.

Writing to a File - The file write operations can be performed by the functions fprintf() and fputs().
fprinf() is used to write formatted string and variable arguments list to the file.
fputs() is used to write a whole line in the file and a newline at the end.

Closing a File - The fclose() function is used to close the file. After successful file operations, it must be closed to remove the file from the memory.
Syntax → fclose(file_pointer);

Example program illustrating File operations.

```
void main()
{
   FILE *fp;
   char str[50], ch[50] = "This is ECE\n";
   fp = fopen("file1.txt","w"); // Create and open the file
   fputs(ch,fp); // Write to File
   printf("%sAbove content is written to file\n",ch);
   fclose(fp); //Close the File
   fp = fopen("file1.txt","r"); // Re-open file to read
   printf("Reading from File.......\n");
   fgets(str, 50, fp); // Read from File
   printf("File Content: %s",str);
   fclose(fp); //Close the File
}
```

Output

```
This is ECE

Above content is written to file

Reading from File.......

File Content: This is ECE
```

RANDOM ACCESS TO FILES
To access records, C language supports random access instead of accessing all the file sequentially. The important functions used in random accessing of files in C language are –

- fseek()
- ftell()
- rewind()

fseek(): This function is used for seeking the pointer position in the file at the specified byte.
Syntax: fseek (file_pointer, displacement, pointer_position);
Here,
file_pointer ---- It is the pointer which points to the file.
displacement ---- It is positive or negative. This is the number of bytes which are skipped backward (if negative) or forward (if positive) from the current position.
pointer_position ---- This sets the pointer position in the file.
Possible Pointer positions are -

Value	Pointer Position
0	Beginning of file.
1	Current position
2	End of file

Examples:
1) fseek(p,10,0) - 0 means pointer position is on beginning of the file, from this statement pointer position is skipped 10 bytes forward from the beginning of the file.

2)fseek(p,5,1) - 1 means current position of the pointer position. From this statement pointer position is skipped 5 bytes forward from the current position.
3)fseek(p, -5,1) - From this statement pointer position is skipped 5 bytes backward from the current position.

ftell(): This function returns the value of the current pointer position in the file. The value is count from the beginning of the file.
Syntax: ftell(fptr); Where ***fptr*** is a file pointer.

rewind(): This function is used to move the file pointer to the beginning of the given file.
Syntax: rewind(fptr); Where ***fptr*** is a file pointer.

Example program to demonstrate random access to files.

```
void main()
{
   FILE *fp;
   char str[50], ch[50] = "This is ECE\n";
   fp = fopen("file1.txt","w+"); // Create and open the file
   fputs(ch,fp); // Write to File
   printf("%sAbove content is written to file\n",ch);
   printf("Current Position=%d\n",ftell(fp));
   printf("Moving File to 5 positions back...\n");
   fseek(fp,-5,1);
   printf("Now Current Position=%d\n",ftell(fp));
   printf("Setting File position to Starting...\n");
   rewind(fp);
   printf("Now Current Position=%d\n",ftell(fp));
   printf("Now Reading from File.......\n");
   fgets(str, 50, fp); // Read from File
   printf("File Content: %s",str);
}
```

Output

```
This is ECE
Above content is written to file
Current Position=12
Moving File to 5 positions back...
Now Current Position=7
Setting File position to Starting...
Now Current Position=0
Now Reading from File.......
File Content: This is ECE
```